Quickstarts

Monitor contract events

Copy page

Track onchain activity by monitoring contract events.

Track contract events and get event logs with the Circle Contracts API.

Prerequisites

You need a deployed contract to monitor. If you completed the **Deploy contracts** tutorial, you can continue with that contract. If your contract was deployed elsewhere, import it in **Step 3**.

Step 1. Update your project

If you haven't already, add run scripts for monitoring contract events to your `package.json`:

```
npm pkg set scripts.webhook="tsx webhook-receiver.ts"
npm pkg set scripts.import-contract="tsx --env-file=.env import-contract.ts"
npm pkg set scripts.create-monitor="tsx --env-file=.env create-monitor.ts"
npm pkg set scripts.get-event-logs="tsx --env-file=.env get-event-logs.ts"
```

If you completed the Deploy contracts tutorial, your project already has the required SDKs installed. The npm scripts previously listed work with your existing setup.

Step 2. Set up your webhook

Event monitors send real-time updates to your webhook endpoint when events happen.

webhook.site ngrok

1. Visit **webhook.site**
2. Copy your unique webhook URL (for example, `https://webhook.site/your-uuid`)

›

Step 3. Register your webhook in Console

Register your webhook URL in the Developer Console:

1. Go to **Developer Console**
2. Navigate to **Webhooks** (left sidebar)
3. Click **Add a webhook**
4. Enter your webhook URL (from Step 1) and create the webhook

> Register your webhook before creating event monitors. This allows Circle to send notifications to your endpoint.

Step 4. Import a contract (optional)

If your contract was deployed elsewhere and is not yet available in the Developer Console, import it first. If you deployed a contract using Circle Contracts, including the **Deploy contracts** tutorial, skip this step. Your contract is already available in the Console.

import-contract.ts import_contract.py

```
import { initiateSmartContractPlatformClient } from "@circle-fin/smart-contrac
                                                  ›
const contractClient = initiateSmartContractPlatformClient({
  apiKey: process.env.CIRCLE_API_KEY,
  entitySecret: process.env.CIRCLE_ENTITY_SECRET,
});

async function importContract() {
  try {
    const response = await contractClient.importContract({
      blockchain: "ARC-TESTNET",
      address: process.env.CONTRACT_ADDRESS,
      name: "MyContract",
    });

    console.log(JSON.stringify(response.data, null, 2));
  } catch (error) {
    console.error("Error importing contract:", error.message);
    throw error;
  }
}

importContract();
```

Run the script:

Node.js Python

```
npm run import-contract
```

> If the contract is already imported, you'll see an error: `contract already exists`. This means the contract is already available in the Console and you can proceed to create an event monitor.

Step 5. Create an event monitor

Event monitors track specific contract events. They send updates to your webhook endpoint. This example monitors `Transfer` events:

create-monitor.ts › create_monitor.py

```
import { initiateSmartContractPlatformClient } from "@circle-fin/smart-contrac

const contractClient = initiateSmartContractPlatformClient({
  apiKey: process.env.CIRCLE_API_KEY,
  entitySecret: process.env.CIRCLE_ENTITY_SECRET,
});

async function createEventMonitor() {
  try {
    const response = await contractClient.createEventMonitor({
      blockchain: "ARC-TESTNET",
      contractAddress: process.env.CONTRACT_ADDRESS,
      eventSignature: "Transfer(address,address,uint256)",
    });

    console.log(JSON.stringify(response.data, null, 2));
  } catch (error) {
    console.error("Error creating event monitor:", error.message);
    throw error;
  }
}

createEventMonitor();
```

Run the script:

Node.js Python

```
npm run create-monitor
```

Response:

```
{
  "eventMonitor": {
    "id": "019bf984-b4da-7026-a3d2-674ce371a933",
    "contractName": "TestERC20Token",
    "contractId": "019bf8be-7be5-7a3e-89cc-05bcd7413f20",
    "contractAddress": "0x281156899e5bd6fecf1c0831ee24894eeeaea2f8",
    "blockchain": "ARC-TESTNET",
    "eventSignature": "Transfer(address,address,uint256)",
    "eventSignatureHash": "0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628
    "isEnabled": true,
    "createDate": "2026-01-26T08:56:22.490638Z",
    "updateDate": "2026-01-26T08:56:22.490638Z"
  }
}
```


Step 6. Receive webhook notifications

When events occur, Circle sends updates to your endpoint. Here is what a `Transfer` event looks like:

```
{
  "subscriptionId": "f0332621-a117-4b7b-bdf0-5c61a4681826",
  "notificationId": "5c5eea9f-398f-426f-a4a5-1bdc28b36d2c",
  "notificationType": "contracts.eventLog",
  "notification": {
    "contractAddress": "0x4abcffb90897fe7ce86ed689d1178076544a021b",
    "blockchain": "ARC-TESTNET",
    "txHash": "0xe15d6dbb50178f60930b8a3e3e775f3c022505ea2e351b6c2c2985d2405c8e
    "userOpHash": "0x78c3e8185ff9abfc7197a8432d9b79566123616c136001e609102c97e7
    "blockHash": "0x0ad6bf57a110d42620defbcb9af98d6223f060de588ed96ae495ddeaf35
    "blockHeight": 22807198,
    "eventSignature": "Transfer(address,address,uint256)",
    "eventSignatureHash": "0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628
    "topics": [
      "0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628f55a4df523b3ef",
      "0x0000000000000000000000000000000000000000000000000000000000000000",
      "0x000000000000000000000000bcf83d3b112cbf43b19904e376dd8dee01fe2758"
    ],
    "data": "0x0000000000000000000000000000000000000000000000000de0b6b3a7640000
    "firstConfirmDate": "2026-01-21T06:53:12Z"
  },
  "timestamp": "2026-01-21T06:53:13.194467201Z",
  "version": 2
}
```


Key fields:

- `notificationType` : Always `"contracts.eventLog"` for event monitor webhooks
- `notification.eventSignature` : The event that was emitted
- `notification.contractAddress` : Address of the contract that emitted the event
- `notification.blockchain` : The blockchain network (for example, `ARC-TESTNET`)
- `notification.txHash` : Transaction hash where the event occurred
- `notification.userOpHash` : User operation hash (for smart contract accounts)
- `notification.blockHash` : Hash of the block containing the transaction
- `notification.blockHeight` : Block number where the event occurred
- `notification.eventSignatureHash` : Keccak256 hash of the event signature
- `notification.topics` : Indexed event parameters (for example, `from` and `to` addresses)

- `notification.data` : Non-indexed event parameters (for example, token amount)
- `notification.firstConfirmDate` : Timestamp when the event was first confirmed
- `timestamp` : Timestamp when the webhook was sent
- `version` : Webhook payload version

You can verify webhook delivery status in the **Developer Console** under Contracts → Monitoring.

Step 7. Retrieve event logs

You can also query event logs with the API. This is useful for past events or if you prefer polling.

Webhooks vs Polling: Webhooks send real-time updates (push). Polling needs periodic API calls (pull). Use webhooks for production and polling for testing or past queries.

get-event-logs.ts get_event_logs.py

```
import { initiateSmartContractPlatformClient } from "@circle-fin/smart-contrac
                                                                            ›
const contractClient = initiateSmartContractPlatformClient({
  apiKey: process.env.CIRCLE_API_KEY,
  entitySecret: process.env.CIRCLE_ENTITY_SECRET,
});

async function getEventLogs() {
  try {
    const response = await contractClient.listEventLogs({
      contractAddress: process.env.CONTRACT_ADDRESS,
      blockchain: "ARC-TESTNET",
      pageSize: 10,
    });

    console.log(JSON.stringify(response.data, null, 2));
  } catch (error) {
    console.error("Error fetching event logs:", error.message);
    throw error;
  }
}

getEventLogs();
```

Run the script:

Node.js Python

```
npm run get-event-logs
```

> Replace `CONTRACT_ADDRESS` with your contract address. You can get this address when you deploy the contract, or by listing your contracts with `listContracts()`.

Response:

```
{
  "eventLogs": [
    {
      "id": "019bf987-f901-7145-9e95-55f177b05b24",
      "subscriptionId": "019bf984-b4da-7026-a3d2-674ce371a933",
      "contractId": "019bf8be-7be5-7a3e-89cc-05bcd7413f20",
      "contractName": "TestERC20Token",
      "blockchain": "ARC-TESTNET",
      "txHash": "0x3bfbab5d5ce0d1a5d682cbc742d3940cf59db0369d173b71ba2a3b8f43bf
      "logIndex": "50",
      "blockHash": "0x7d12148f9331556b31f84f58a41b7ff16eaaa47940f9e86733037d7ab
      "blockHeight": 23686153,
      "contractAddress": "0x281156899e5bd6fecf1c0831ee24894eeeaea2f8",
      "eventSignature": "Transfer(address,address,uint256)",
      "eventSignatureHash": "0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a116
      "topics": [
        "0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628f55a4df523b3ef",
        "0x0000000000000000000000000000000000000000000000000000000000000000",
        "0x000000000000000000000000bcf83d3b112cbf43b19904e376dd8dee01fe2758"
      ],
      "data": "0x0000000000000000000000000000000000000000000000000de0b6b3a76400
      "decodedTopics": null,
      "decodedData": null,
      "userOpHash": "0x66befac1a371fcdddf1566215e4677127e111dff9253f306f7096fec
      "firstConfirmDate": "2026-01-26T08:59:55Z",
      "createDate": "2026-01-26T08:59:56.545962Z",
      "updateDate": "2026-01-26T08:59:56.545962Z"
    }
  ]
}
```


You can view, update, and delete event monitors with the Circle Contracts API. See the **API Reference** for details on managing your monitors.

Summary

After completing this tutorial, you've successfully:

- Set up webhook endpoints using webhook.site or `ngrok`

- Registered webhooks in the Developer Console
- Created event monitors for specific contract events
- Received real-time webhook updates for contract events
- Retrieved past event logs with the Circle SDK

Was this page helpful? Yes No

Interact with contracts Register your first AI agent

Links

Network status

Blog

Community

Arc homepage

Resources

Terms of service

Privacy policy

Cookie policy

Powered by mintlify

By using the Arc Network Testnet, you agree to the Terms of Service and Privacy Policy.